

GALAXY SWISS BOURDIN

DOCUMENTATION TECHNIQUE

Application de gestion des frais de déplacement

· Java Swing · MySQL ·

BTS SIO – Épreuve E6 · Présentation de projet

Sommaire

Sommaire	2
1. Présentation générale du projet	3
1.1 Contexte	3
1.2 Architecture logicielle – Modèle MVC en couches	3
1.3 Schéma de la base de données	3
2. Diagramme de classes simplifié	5
3. Couche BDD	6
3.1 connexion.java – Singleton JDBC	6
Méthode getInstance()	6
3.2 AccesData.java – Façade d'accès	6
Méthodes de gestion des utilisateurs	6
Méthodes statistiques	7
4. Couche DAO	8
4.1 Classe abstraite DAO<T>	8
4.2 utilisateurDAO – Gestion des utilisateurs	8
Authentification – connexionAppli()	8
Récupération de tous les visiteurs – findall()	8
Modification des coordonnées – modifCoordonnees()	9
Création d'un visiteur – crerUnVisiteur()	9
Suppression – supprimerVisiteur()	9
4.3 lignefraishorsforfaitDAO – Statistiques FHF	9
getStatsByRegionAndMois() – Nb de lignes FHF par visiteur	9
getMontantByRegionAndMois() – Montant total FHF par visiteur	10
getStatsMoyMontantByMois() – Moyenne des montants FHF par région	10
4.4 fraisforfaitDAO – Statistiques forfait	10
getTotalFraisForfaitByMoisAndRegion()	10
4.5 lignefraisforfaitDAO – Moyenne forfait par région	11
getStatsMoyMontantByMois()	11
5. Couche INTERFACE	12
5.1 Pattern commun à toutes les fenêtres	12
5.2 connexionINTERFACE – Écran d'authentification	12
Logique du bouton « Se connecter »	12
5.3 accueilINTERFACE – Menu principal	13
5.4 selectionnerVisiteurINTERFACE – Liste et recherche des visiteurs	13
Composants clés	13
Logique de filtrage – rechercherVisiteur()	13
5.5 modifiercoordonneeINTERFACE – Formulaire de modification	14
Logique du bouton « Confirmer »	14
5.6 creationVisiteurINTERFACE – Formulaire de création	14
5.7 supprimerVisiteurINTERFACE – Suppression d'un visiteur	15
Gestion des cas d'erreur	15
6. Module Statistiques	16
6.1 accueilSTATINTERFACE – Menu des statistiques	16
6.2 Structure commune aux cinq écrans	16

Logique commune – méthode chargerStats()	17
Différences entre les cinq écrans	17
7. Justification des choix techniques	18
8. Flux de navigation	19

1. Présentation générale du projet

1.1 Contexte

L'application GSB (Galaxy Swiss Bourdin) est un outil interne destiné au personnel administratif du laboratoire pharmaceutique éponyme. Elle permet de gérer les fiches de frais de déplacement soumises par les visiteurs médicaux : consultation, création, modification et suppression des visiteurs, ainsi que la visualisation de statistiques de remboursement.

L'accès à l'application est restreint aux profils RH, DI et RF. Les visiteurs médicaux (rôle VS) n'ont pas accès à cette interface de gestion.

1.2 Architecture logicielle – Modèle MVC en couches

Le projet suit une architecture en couches inspirée du patron MVC (Modèle – Vue – Contrôleur), déclinée en quatre packages Java distincts :

Package	Rôle dans l'architecture
BDD	Connexion JDBC et façade d'accès aux données (AccesData). Joue le rôle de Contrôleur.
DAO	Couche d'accès aux données : une classe par entité, encapsule toutes les requêtes SQL. Fait partie du Modèle.
POJO	Objets métier simples (entités Java correspondant aux tables SQL). Représentent le Modèle.
INTERFACE	Fenêtres graphiques Swing (JFrame). Constituent la Vue.
MAIN	Point d'entrée unique de l'application (classe programme).

Ce découpage garantit une **séparation des responsabilités** : les interfaces ne contiennent aucune requête SQL directe — elles passent systématiquement par AccesData, qui délègue aux DAO.

1.3 Schéma de la base de données

La base `gsbfrais-2025` contient six tables principales. Les relations entre elles sont les suivantes :

utilisateur

PK idUtilisateur (VARCHAR)
nom, prenom, login, mdp, idrole (FK→role), adresse, cp, ville, dateEmbauche, email, telFixe, telPortable, region

role

PK idRole (VARCHAR) | libelle

fichefrais

PK idFiche (LONG) | FK idVisiteur → utilisateur
annee, mois, nbJustificatifs, montantValide, dateModif, idEtat

lignefraishorsforfait

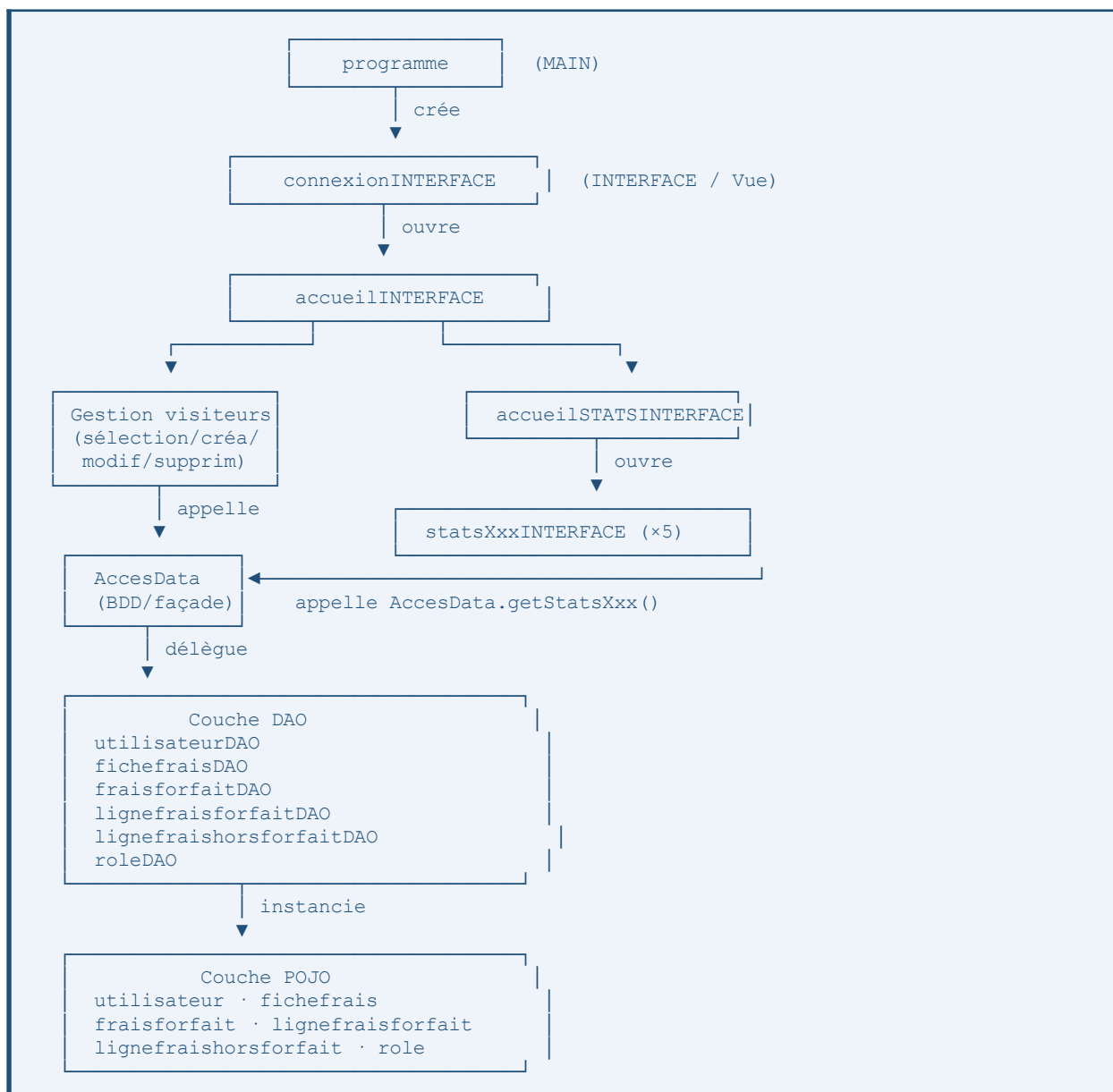
PK idLigneFHF | FK idFiche → fichefrais

```
libelle, dateFrais, montant (DECIMAL)
lignefraisforfait
  PK (idFiche, idFraisForfait) | FK idFiche → fichefrais
                                | FK idFraisForfait → fraisforfait
  quantite (INT)
fraisforfait
  PK idFraisForfait (VARCHAR) | libelle, montant unitaire (DECIMAL)
```

 **Relation centrale** — Un utilisateur possède 0..N fiches de frais (fichefrais). Chaque fiche est décomposée en lignes hors-forfait (lignefraishorsforfait) et en lignes forfait (lignefraisforfait, référençant un type de frais prédéfini).

2. Diagramme de classes simplifié

Le diagramme ci-dessous représente les dépendances entre les classes principales du projet :



i Choix technique — L'interface ne manipule jamais les DAO directement. Elle passe toujours par **AccesData**, ce qui centralise la logique d'accès et facilite la maintenance : si une requête change, seul **AccesData** ou le DAO concerné est modifié.

3. Couche BDD

3.1 connexion.java – Singleton JDBC

Cette classe gère l'unique connexion JDBC vers la base de données MySQL. Elle implémente un pattern pseudo-singleton : la connexion est stockée dans un attribut statique et retournée à chaque appel.

Attribut / Constante	Valeur
URL	jdbc:mysql://localhost:3306/gsbfrais-2025
UTILISATEUR	myroot
MOT_DE_PASSE	root123*
con (static)	Instance unique de Connection

Méthode getInstance()

Établit la connexion via `DriverManager.getConnection()` et la retourne. Si une exception SQL survient, elle est capturée et affichée dans la console (`e.printStackTrace()`).

```
public static Connection getInstance() {
    try {
        con = DriverManager.getConnection(URL, UTILISATEUR, MOT_DE_PASSE);
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return con;
}
```

 **Limite identifiée** — La connexion est recrée à chaque appel de `getInstance()` au lieu d'être réutilisée si elle est encore valide. Dans un contexte de production, on utiliserait un pool de connexions (ex. HikariCP) pour améliorer les performances.

3.2 AccesData.java – Façade d'accès

AccesData est le seul point de contact entre les interfaces graphiques et la base de données. Elle instancie les DAO à la demande et leur délègue les opérations. La connexion est initialisée statiquement au chargement de la classe.

```
static Connection con = connexion.getInstance();
```

Toutes ses méthodes sont **statiques** (sauf `connexionAppli` et `crerUnVisiteur`), ce qui permet aux interfaces de les appeler directement sans instancier AccesData.

Méthodes de gestion des utilisateurs

Signature	Rôle	Retour
-----------	------	--------

<code>getUnUtilisateur(String id)</code>	Retourne un objet utilisateur complet à partir de son identifiant	<code>utilisateur</code>
<code>getDesUtilisateur()</code>	Retourne tous les visiteurs (idrole = 'VS')	<code>ArrayList<utilisateur></code>
<code>modifierCoordonnees(id, nom, prenom, telFixe, telPortable, region, adresse)</code>	Met à jour les coordonnées via UPDATE SQL	<code>boolean</code>
<code>connexionAppli(login, mdp)</code>	Vérifie les identifiants et retourne le rôle ou null	<code>String</code>
<code>crerUnVisiteur(...14 params...)</code>	Insère un nouveau visiteur (INSERT)	<code>boolean</code>
<code>supprimerUnVisiteur(id)</code>	Supprime un visiteur (DELETE)	<code>boolean</code>
<code>getLesRegions()</code>	Retourne les régions distinctes pour les listes déroulantes	<code>ArrayList<String></code>

Méthodes statistiques

Signature	Rôle	Retour
<code>getStatsLigneFHF(region, mois)</code>	Nb de lignes FHF par visiteur (région + mois)	<code>ArrayList<Object[]></code>
<code>getStatsMontantFHF(region, mois)</code>	Montant total FHF par visiteur (région + mois)	<code>ArrayList<Object[]></code>
<code>getStatsMoyMontantFHF(mois)</code>	Moyenne des montants FHF par région pour un mois	<code>ArrayList<Object[]></code>
<code>getStatsMoyMontantForfait(mois)</code>	Moyenne des montants forfait par région pour un mois	<code>ArrayList<Object[]></code>
<code>getTotalFraisForfaitByMoisAndRegion(mois, region)</code>	Total forfait (quantité × montant unitaire) par visiteur	<code>ArrayList<Object[]></code>

4. Couche DAO

4.1 Classe abstraite DAO<T>

Toutes les classes DAO héritent de la classe générique `DAO<T>`. Elle définit le contrat CRUD commun et stocke la connexion dans un attribut statique protégé.

```
public abstract class DAO<T> {
    protected static Connection connect = null;
    public DAO(Connection conn) { this.connect = conn; }

    public abstract boolean create(T obj);
    public abstract boolean delete(T obj);
    public abstract boolean update(T obj);
    public abstract T find(int id);
}
```

i Choix technique — L'utilisation d'un générique <T> évite la duplication de code : chaque DAO hérite des méthodes CRUD sans les réécrire. Dans cette version du projet, seules les méthodes `find()` et `findall()` sont réellement implémentées ; `create()`, `delete()` et `update()` retournent `false` (développement en cours).

4.2 utilisateurDAO – Gestion des utilisateurs

Ce DAO est le plus complet. Il contient toutes les opérations métier liées à la table `utilisateur` : authentification, CRUD visiteurs, et récupération des régions.

Authentification – connexionAppli()

Utilise une requête préparée (**PreparedStatement**) pour éviter toute injection SQL. Retourne le rôle si les identifiants correspondent, `null` sinon.

```
SELECT idrole
FROM utilisateur
WHERE login = ? -- paramètre 1 : login saisi
AND mdp = ? -- paramètre 2 : mot de passe saisi
```

🔒 Sécurité — Les requêtes paramétrées (PreparedStatement) protègent contre les injections SQL : les valeurs saisies par l'utilisateur sont transmises comme paramètres et non concaténées dans la requête.

Récupération de tous les visiteurs – findAll()

Filtre sur le rôle 'VS' pour ne retourner que les visiteurs médicaux, pas les administrateurs.

```
SELECT * FROM utilisateur WHERE idrole = 'VS'
```

Modification des coordonnées – modifCoordonnees()

Met à jour les champs modifiables (adresse, téléphones, région, nom, prénom) via un UPDATE paramétré.

```
UPDATE utilisateur
SET    nom=?, prenom=?, telFixe=?, telPortable=?,
      region=?, adresse=?
WHERE  idUtilisateur=?
```

Création d'un visiteur – crerUnVisiteur()

INSERT complet avec 14 paramètres. Le rôle est forcé à 'vs' indépendamment de la valeur passée en argument.

```
INSERT INTO utilisateur
(idUtilisateur, nom, prenom, login, mdp, idrole,
 adresse, cp, ville, dateEmbauche, email, telFixe, telPortable, region)
VALUES (?, ?, ?, ?, ?, 'VS', ?, ?, ?, ?, ?, ?, ?, ?)
```

Suppression – supprimerVisiteur()

DELETE simple par identifiant. Si le visiteur possède encore des fiches de frais non soldées, la contrainte de clé étrangère MySQL lève une exception SQL, capturée et retournée sous forme de false.

```
DELETE FROM utilisateur WHERE idUtilisateur = ?
```

4.3 lignefraishorsforfaitDAO – Statistiques FHF

Ce DAO contient les trois requêtes statistiques les plus complexes du projet, toutes basées sur des jointures multi-tables.

getStatsByRegionAndMois() – Nb de lignes FHF par visiteur

Pour chaque visiteur d'une région et d'un mois donnés, compte le nombre de lignes de frais hors forfait déclarées.

```
SELECT    u.nom, u.prenom,
          COUNT(fhf.idLigneFHF) AS nb_lignes
FROM      utilisateur u
JOIN      fichefrais ff      ON ff.idVisiteur = u.idUtilisateur
JOIN      lignefraishorsforfait fhf ON fhf.idFiche = ff.idFiche
WHERE     u.region      = ?  -- filtre région
AND       ff.mois       = ?  -- filtre mois
GROUP BY u.idUtilisateur, u.nom, u.prenom
ORDER BY u.nom, u.prenom
```

 **Logique** — La jointure en trois tables (utilisateur → fichefrais → lignefraishorsforfait) est nécessaire car la région et l'identité du visiteur sont dans utilisateur, la date (mois) est dans fichefrais, et les lignes de dépenses sont dans lignefraishorsforfait.

getMontantByRegionAndMois() – Montant total FHF par visiteur

Calcule la somme des montants (`SUM`) de toutes les lignes FHF pour chaque visiteur d'une région et d'un mois. Filtre supplémentaire sur le rôle VS.

```
SELECT    u.nom, u.prenom,
          SUM(fhf.montant) AS montant_total
FROM      utilisateur u
JOIN      fichefrais ff      ON ff.idVisiteur = u.idUtilisateur
JOIN      lignefraishorsforfait fhf ON fhf.idFiche = ff.idFiche
WHERE     u.region      = ?
          AND ff.mois    = ?
          AND u.idRole   = 'VS'
GROUP BY  u.idUtilisateur, u.nom, u.prenom
ORDER BY  u.nom, u.prenom
```

getStatsMoyMontantByMois() – Moyenne des montants FHF par région

Requête utilisant `AVG` et `COUNT(DISTINCT)` pour obtenir, par région, la moyenne des montants FHF et le nombre de visiteurs distincts ayant soumis une fiche sur le mois sélectionné.

```
SELECT    u.region,
          COUNT(DISTINCT ff.idVisiteur) AS nb_visiteurs,
          AVG(fhf.montant)              AS moyenne_montant_FHF
FROM      lignefraishorsforfait fhf
JOIN      fichefrais ff ON fhf.idFiche      = ff.idFiche
JOIN      utilisateur u  ON ff.idVisiteur   = u.idUtilisateur
WHERE     ff.mois = ?
GROUP BY  u.region
ORDER BY  u.region
```

4.4 fraisforfaitDAO – Statistiques forfait

getTotalFraisForfaitByMoisAndRegion()

Calcule le total des frais forfait pour chaque visiteur en multipliant la quantité par le montant unitaire du type de frais. Nécessite une jointure supplémentaire sur `fraisforfait` pour récupérer les tarifs.

```
SELECT    u.nom, u.prenom,
          SUM(lff.quantite * ff.montant) AS total_forfait
FROM      lignefraisforfait lff
JOIN      fraisforfait ff ON lff.idFraisForfait = ff.idFraisForfait
JOIN      fichefrais fiche ON lff.idFiche      = fiche.idFiche
JOIN      utilisateur u   ON fiche.idVisiteur = u.idUtilisateur
WHERE     fiche.mois      = ?
          AND u.region    = ?
GROUP BY  fiche.idVisiteur, u.nom, u.prenom
```

```
ORDER BY u.nom, u.prenom
```

4.5 lignefraisforfaitDAO – Moyenne forfait par région

getStatsMoyMontantByMois()

Requête avec **sous-requête** : calcule d'abord le total forfait par visiteur, puis fait la moyenne par région. Deux passages du paramètre mois sont nécessaires (un dans la sous-requête, un dans la requête principale).

```
SELECT    u.region,
          COUNT(DISTINCT ff.idVisiteur) AS nb_visiteurs,
          AVG(sq.total_forfait)        AS moyenne_montant_forfait
FROM (
  -- Sous-requête : total forfait par visiteur
  SELECT  ff2.idVisiteur,
          SUM(lff.quantite * f.montant) AS total_forfait
  FROM    lignefraisforfait lff
  JOIN    fraisforfait f    ON lff.idFraisForfait = f.idFraisForfait
  JOIN    fichefrais ff2   ON lff.idFiche        = ff2.idFiche
  WHERE   ff2.mois = ?
  GROUP BY ff2.idVisiteur
) AS sq
JOIN fichefrais ff ON sq.idVisiteur = ff.idVisiteur
JOIN utilisateur u  ON ff.idVisiteur = u.idUtilisateur
WHERE ff.mois = ?
GROUP BY u.region
ORDER BY u.region
```

i Justification de la sous-requête — Il est impossible de faire AVG(SUM(...)) directement en SQL. On calcule d'abord le total par visiteur dans une sous-requête, puis on applique AVG sur ce résultat dans la requête externe.

5. Couche INTERFACE

5.1 Pattern commun à toutes les fenêtres

Toutes les interfaces héritent de `JFrame` et respectent le même patron de conception :

Convention	Description
Layout null	Positionnement absolu des composants (<code>setBounds</code>). Simplifie la mise en page mais la rend non responsive.
Lazy init (<code>getXxx()</code>)	Chaque composant est initialisé dans sa méthode getter avec vérification de nullité : <code>if (btn == null) { btn = new JButton(); } return btn;</code>
Propagation du contexte	Le login et le rôle de l'utilisateur connecté sont transmis de fenêtre en fenêtre via le constructeur (<code>String login, String role</code>).
Navigation	Chaque ouverture de fenêtre suit le même pattern : <code>new FenetreXxx(login, role).setVisible(true); dispose();</code>
Taille standard	Toutes les fenêtres font 950 × 600 px.

5.2 connexionINTERFACE – Écran d'authentification

Première fenêtre affichée. Permet la saisie du login et du mot de passe. Gère l'authentification et le routage vers l'accueil selon le rôle.

Composant	Description
<code>JTextField (txtLogin)</code>	Saisie du login utilisateur
<code>JPasswordField (txtMotDePasse)</code>	Saisie masquée du mot de passe
<code>JButton (Se connecter)</code>	Déclencheur de l'authentification — défini comme bouton par défaut (Entrée clavier)
<code>JLabel (logoGSB)</code>	Logo GSB chargé depuis le dossier <code>/image/GSB.png</code> , redimensionné à 199×83 px via <code>Image.getScaledInstance()</code>

Logique du bouton « Se connecter »

```
String login = txtLogin.getText();
String mdp   = txtMotDePasse.getText();
AccesData ad = new AccesData();
String role  = ad.connexionAppli(login, mdp);

if (role != null && Set.of("RH","DI","RF").contains(role)) {
    // Succès : ouverture du menu principal
    new accueilINTERFACE(login, role).setVisible(true);
    dispose();
} else if (role != null) {
    // Rôle VS ou inconnu : accès refusé
    JOptionPane.showMessageDialog(null,
        "Vous n'avez pas le droit d'accéder à l'application !");
} else {
```

```
// Identifiants incorrects
JOptionPane.showMessageDialog(null,
    "Login ou mot de passe incorrect !");
}
```

5.3 accueilINTERFACE – Menu principal

Affiché après connexion réussie. Centralise la navigation vers toutes les fonctionnalités. Affiche le login et le rôle de l'utilisateur connecté dans un panneau dédié.

Zone	Contenu
Panel info utilisateur (haut gauche)	Affiche le login et le rôle via deux JLabel (mis à jour dans le constructeur avec setText())
Boutons de navigation	Gestion des visiteurs, statistiques, etc. — selon les droits du rôle
Bouton « Se déconnecter »	Réouvre connexionINTERFACE et ferme la fenêtre courante

5.4 selectionnerVisiteurINTERFACE – Liste et recherche des visiteurs

Affiche la liste complète des visiteurs dans un tableau triable. Permet de filtrer par nom/prénom, identifiant ou région.

Composants clés

Composant	Rôle
JTable + DefaultTableModel	Affiche les colonnes Identifiant, Nom, Prénom, Région
TableRowSorter	Tri dynamique par clic sur l'en-tête de colonne (tri initial par identifiant)
3 × JTextField (filtres)	Filtres Nom/Prénom, Identifiant, Région — filtrage côté client (en mémoire)
MouseListener sur la table	Au clic sur une ligne : récupère l'ID, charge l'objet utilisateur via <code>AccesData.getUnUtilisateur()</code> , ouvre <code>utilisateurINTERFACE</code>

Logique de filtrage – `rechercherVisiteur()`

La liste complète est chargée depuis la base, puis filtrée en mémoire selon les saisies. Le filtrage est insensible à la casse (`.toLowerCase()`).

```
ArrayList<utilisateur> liste = AccesData.getDesUtilisateur();
for (utilisateur u : liste) {
    if ((nomPrenom.isEmpty() || u.getNom().toLowerCase().contains(nomPrenom)
        || u.getPrenom().toLowerCase().contains(nomPrenom))
        && (identifiant.isEmpty() || u.getIdUtilisateur().toLowerCase().contains(identifiant))
        && (region.isEmpty() || u.getRegion().toLowerCase().contains(region))) {
        model.addRow(new Object[]{ u.getIdUtilisateur(), u.getNom(), ... });
    }
}
```

}

i Choix technique — Le filtrage est réalisé côté client (en Java) plutôt qu'en SQL. Cela simplifie le code et évite des requêtes répétées, mais serait à revoir pour une base avec de nombreux visiteurs (filtrage SQL préférable pour la scalabilité).

5.5 modifiercoordonneeINTERFACE – Formulaire de modification

Permet de modifier les coordonnées d'un visiteur sélectionné. Le nom et le prénom sont affichés en lecture seule (JLabel avec bordure grise) ; seuls l'adresse, les téléphones et la région sont modifiables.

Composant	Rôle
JLabel (lblValeurNom/Prenom)	Nom et prénom en lecture seule — style grisé avec bordure LineBorder
JTextField (adresse, fixe, portable, region)	Champs modifiables, pré-remplis dans le constructeur avec les valeurs actuelles du visiteur
JButton « Confirmer »	Appelle AccesData.modifierCoordonnees() — affiche un JOptionPane de succès ou d'erreur
JButton « Annuler »	Retour direct à l'accueil sans modification

Logique du bouton « Confirmer »

```
// Récupération des valeurs saisies
String adresse = zonetexteAdresse.getText().trim();
String fixe = zonetexteFixe.getText().trim();
String portable = zonetextePortable.getText().trim();
String region = zonetexteRegion.getText().trim();
// Nom/prénom : non modifiables, tirés de l'objet métier
String nom = visiteurSelectionne.getNom();
String prenom = visiteurSelectionne.getPrenom();

boolean ok = AccesData.modifierCoordonnees(
    id, nom, prenom, fixe, portable, region, adresse);
if (ok) { JOptionPane → "Modification réussie" + retour liste; }
else { JOptionPane → "Utilisateur introuvable."; }
```

5.6 creationVisiteurINTERFACE – Formulaire de création

Formulaire complet de 13 champs permettant d'enregistrer un nouveau visiteur. Le rôle est forcé à 'VS' en dur dans l'appel à AccesData.

Champ	Type de composant
ID, Nom, Prénom, Login	JTextField
Mot de passe	JPasswordField (saisie masquée)
Adresse, CP, Ville, Email, Fixe, Portable, Région	JTextField

Date d'embauche	JTextField avec placeholder "AAAA-MM-JJ"
------------------------	--

Après insertion réussie, la méthode `viderChamps()` remet tous les champs à vide pour permettre la saisie d'un autre visiteur sans fermer la fenêtre.

⚠ Limite identifiée — Aucune validation de saisie n'est implémentée côté interface (format de date, format email, doublon d'identifiant). En cas d'erreur (clé dupliquée, contrainte SQL), l'exception est capturée et l'interface affiche simplement "Erreur !".

5.7 supprimerVisiteurINTERFACE – Suppression d'un visiteur

Reprend la même liste filtrée que l'écran de sélection. La suppression est protégée par une double confirmation.

Composant	Rôle
JTable + filtres (id, nom, région)	Même logique de recherche que sélectionnerVisiteurINTERFACE
MouseListener	Au clic : mémorise l'ID dans <code>idSelectionne</code> et active le bouton Supprimer
JButton « Supprimer » (désactivé par défaut)	Activé seulement après sélection d'une ligne — <code>setEnabled(false/true)</code>
JOptionPane.showConfirmDialog	Demande confirmation avant d'exécuter la suppression

Gestion des cas d'erreur

Si la suppression échoue (fiche de frais existante → contrainte FK MySQL), un message d'erreur explicite est affiché à l'utilisateur :

```
boolean succes = AccesData.supprimerUnVisiteur(idSelectionne);
if (succes) {
    JOptionPane → "Le visiteur a bien été supprimé !" (icône info ✓)
} else {
    JOptionPane → "Impossible de supprimer : le visiteur possède
encore des fiches frais non remboursées..." (icône erreur
✗)
}
// Dans tous les cas : reset de la sélection + rechargement de la liste
idSelectionne = null;
btnSupprimer.setEnabled(false);
rechercherVisiteur();
```


6. Module Statistiques

6.1 accueilSTATSINTERFACE – Menu des statistiques

Écran intermédiaire accessible depuis le menu principal. Propose cinq boutons de navigation vers les différents écrans statistiques.

Bouton	Écran cible
Stats NbFicheFHF/Region/Mois/Visiteur	statsLigneFHF
Stats MontantFHF/Region/Mois/Visiteur	statsMontantFHF
Stats MontantFF/Region/Mois/Visiteur	statsMontantFFRegionMoisVisiteur
Stats MoyenneMontantFF/Mois/Region	statsMoyMontantFFMoisRegionINTERFACE
Stats MoyenneMontantFHF/Mois/Region	StatsMoyMontantFHFMoisRegionINTERFACE

6.2 Structure commune aux cinq écrans

Tous les écrans de statistiques partagent exactement la même architecture. Seuls changent le titre, les colonnes du tableau et l'appel à AccesData.

Élément	Description
Titre (JLabel, Arial Bold 25pt)	Centré en haut de la fenêtre, décrit la statistique affichée
ComboBox Région	Chargée dynamiquement via AccesData.getLesRegions() (SELECT DISTINCT region). Absente sur les écrans de moyenne globale.
ComboBox Mois	12 entrées nommées en français (tableau NOMS_MOIS[]), index 0 ignoré
Bouton « Afficher »	Déclenche chargerStats() — efface le tableau puis le recharge
JTable (non éditable)	DefaultTableModel avec isCellEditable() retournant false. Cellules centrées via DefaultTableCellRenderer.
JScrollPane	Encapsule le JTable pour permettre le défilement si les résultats sont nombreux
Label « Aucun résultat » (rouge, italic)	Visible uniquement si la requête retourne une liste vide (scénario alternatif)
Bouton « Retour »	Retourne à accueilSTATSINTERFACE en transmettant login et role
Bouton « Se déconnecter »	Retourne à connexionINTERFACE

Logique commune – méthode chargerStats()

```
private void chargerStats() {
    model.setRowCount(0); // Vide le tableau
    lblAucunResultat.setVisible(false);

    String region = (String) comboRegion.getSelectedItem();
    BigDecimal mois = BigDecimal.valueOf(
        comboMois.getSelectedIndex() + 1);

    if (region == null) return;

    ArrayList<Object[]> resultats =
        AccesData.getStatsXxx(region, mois); // ← méthode spécifique

    if (resultats.isEmpty()) {
        lblAucunResultat.setVisible(true); // Scénario alternatif
    } else {
        for (Object[] ligne : resultats)
            model.addRow(ligne);
    }
}
```

Différences entre les cinq écrans

Écran	Méthode AccesData	Filtres	Colonnes JTable
statsLigneFHF	getStatsLigneFHF()	Région + Mois	Nom · Prénom · Nb lignes FHF
statsMontantFHF	getStatsMontantFHF()	Région + Mois	Nom · Prénom · Montant total
statsMontantFFRegionMois	getTotalFraisForfaitByMoisAndRegion()	Région + Mois	Nom · Prénom · Total forfait
statsMoyMontantFFMoisRegion	getStatsMoyMontantForfait()	Mois uniquement	Région · Nb visiteurs · Moyenne FF
StatsMoyMontantFHFMoisRegion	getStatsMoyMontantFHF()	Mois uniquement	Région · Nb visiteurs · Moyenne FHF

7. Justification des choix techniques

Choix technique	Justification
Java Swing	Framework graphique natif Java, ne nécessite pas de serveur web. Adapté à une application desktop interne d'entreprise.
Architecture en couches (MVC)	Séparation claire entre données (POJO/DAO), logique métier (AccesData) et présentation (INTERFACE). Facilite la maintenance et les évolutions.
PreparedStatement	Protection contre les injections SQL. Toutes les requêtes paramétrées utilisent des ? au lieu de concaténations de chaînes.
ArrayList<Object[]> pour les stats	Choix pragmatique : les requêtes statistiques retournent des agrégats qui ne correspondent pas directement à un POJO existant. Object[] permet de transporter des données hétérogènes sans créer de classes supplémentaires.
Filtrage côté client (recherche visiteurs)	Simplifie le code et évite des allers-retours réseau. Acceptable pour un nombre de visiteurs raisonnable.
BigDecimal pour le mois	Cohérence avec le type DECIMAL de la colonne mois en base MySQL. Évite les erreurs d'arrondi liées aux float/double.
Bouton Supprimer désactivé par défaut	Empêche une suppression accidentelle sans sélection préalable. Bonne pratique UX.

8. Flux de navigation

```

programme.main()
└─> connexionINTERFACE
    ├── [identifiants invalides] → JOptionPane erreur
    ├── [rôle VS] → JOptionPane accès refusé
    └─> accueilINTERFACE (login, role)
        ├── selectionnerVisiteurINTERFACE
        │   └─> [clic ligne] → utilisateurINTERFACE
        │       └─> modifiercoordonneeINTERFACE
        │           ├── [Confirmer] → selectionnerVisiteurINTERFACE
        │           └─> [Annuler] → accueilINTERFACE
        ├── creationVisiteurINTERFACE
        │   ├── [Inscrire] → viderChamps() (reste sur la page)
        │   └─> [Retour] → accueilINTERFACE
        ├── supprimerVisiteurINTERFACE
        │   └─> [Supprimer + confirm] → rechercherVisiteur() (reste sur la page)
        └─> accueilSTATSINTERFACE
            ├── statsLigneFHF
            ├── statsMontantFHF
            ├── statsMontantFFRegionMoisVisiteur
            ├── statsMoyMontantFFMoisRegionINTERFACE
            └─> StatsMoyMontantFHFMoisRegionINTERFACE

[Se déconnecter] depuis n'importe quelle fenêtre → connexionINTERFACE
[Retour] depuis n'importe quelle fenêtre → fenêtre parente
  
```